Computers & Security 154 (2025) 104457

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

An experimental evaluation of TEE technology: Benchmarking transparent approaches based on SGX, SEV, and TDX

Luigi Coppolino, Salvatore D'Antonio, Giovanni Mazzeo *, Luigi Romano

University of Naples 'Parthenope', Centro Direzionale Isola E2, Naples, 80100, Italy

ARTICLE INFO

Keywords:
Trusted Execution Environment
Confidential computing
AMD SEV
Intel TDX
Intel SGX
Gramine
Occlum

ABSTRACT

Protection of data-in-use is a key priority, for which Trusted Execution Environment (TEE) technology has unarguably emerged as a — possibly the most — promising solution. Multiple server-side TEE offerings have been released over the years, exhibiting substantial differences with respect to several aspects. The first comer was Intel SGX, which featured *Process-based TEE* protection, an efficient yet difficult to use approach. Some SGX limitations were (partially) overcome by runtimes, notably: *Gramine*, *Scone*, and *Occlum*. A major paradigm shift was later brought by AMD SEV, with *VM-based TEE* protection, which enabled "lift-and-shift" deployment of legacy applications. This new paradigm has been implemented by Intel only recently, in TDX. While the threat model of the aforementioned TEE solutions has been widely discussed, a thorough performance comparison is still lacking in the literature. This paper provides a comparative evaluation of *TDX*, *SEV*, *Gramine-SGX*, and *Occlum-SGX*. We study computational overhead and resource usage, under different operational scenarios and using a diverse suite of legacy applications. By doing so, we provide a reliable performance assessment under realistic conditions. We explicitly emphasize that — at the time of writing — TDX was recently released to the public. Thus, the evaluation of TDX is a unique feature of this study.

1. Introduction

Trusted Execution Environments (TEEs) offer better performance than alternatives like Homomorphic Encryption or Secure Multi-Party Computation (Coppolino et al., 2019), making them a key technology for secure computing. Initial *Process-based TEE* solutions, such as Intel *Software Guard eXtensions* (SGX) (McKeen et al., 2016), enabled data-in-use protection in untrusted cloud platforms. However, researchers and practitioners identified limitations, particularly memory constraints and programming restrictions, which made adapting legacy software to SGX challenging and error-prone. The enrichment of the Intel SGX technology with a runtime layer — e.g., *Gramine* (Tsai et al., 2017) (formerly known as *Graphene*), *Occlum* (Shen et al., 2020), or *Scone* (Arnautov et al., 2016), *Veracruz* (Brossard et al., 2022), *Enarx* (Anon, 2025) — helped to mitigate porting issues but at the cost of a larger Trusted Computing Base (TCB). AMD with the *Secure Encrypted Virtualization* (SEV) (Kaplan et al., 2016) technology introduced the concept of a *VM-based TEE*, which is significantly more user-friendly, since it allows existing applications to run in the secure environment without any modification. The downside, as compared to Process-based TEE, is a weaker threat model (see Fig. 1). A VM-based TEE has been recently presented by Intel, too. It is called *Trust Domain eXtension* (TDX) (Anon, 2023a), and builds on lessons learned from SGX.

While the threat models of these technologies (see Fig. 3) are well known, and detailed analyses of the tradeoffs of alternative solutions have been made (Sahita et al., 2021; Hetzelt and Buhren, 2017; Fei et al., 2021), the scientific/technical literature provides limited coverage of performance evaluation of TEE offerings, since the currently available comparison of existing TEE approaches for transparent — or quasi-transparent — protection of data-in-use from a quantitative point of view is largely incomplete. This undermines the possibility for security engineers/researchers to take informed decisions about the specific TEE solution to use, based on individual application requirements.

There are some previous research works featuring comparative analyses of TEE solutions (Göttel et al., 2018; Akram et al., 2021; Mofrad et al., 2018), which mainly focused on setting side by side SGX and SEV. In just one case, *Gramine-SGX* was also included in the evaluation. No experimental evaluation exists to date on TDX, since this technology has only recently been released.

In this work, we delve into a comprehensive comparative analysis of a wide spectrum of solutions for transparent TEE support, ranging from earlier proposals (namely: *Gramine-SGX* and *Occlum-SGX*) to the most recent one (namely: *TDX*). To the best of our knowledge, this is one of the first papers to provide an experimental evaluation of TDX (as already mentioned, TDX was recently released to the public).

* Corresponding author.
E-mail address: giovanni.mazzeo@uniparthenope.it (G. Mazzeo).

https://doi.org/10.1016/j.cose.2025.104457
Received 4 November 2024; Received in revised form 28 February 2025; Accepted 20 March 2025
Available online 31 March 2025
0167-4048/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

The motivation behind choosing *Gramine* and *Occlum* runtimes is based on their widespread use. In fact, from our research in Google Scholar the *Gramine*, *Occlum*, *Scone*, *Veracruz*, and *Enarx* keywords appear 1308, 432, 1045, 94, 78 times, respectively. Since *Scone* is not open-source, we selected *Gramine* and *Occlum* given that are among the most popular open-source runtimes.

We investigate the performance tradeoffs of alternative TEE solutions, with respect to the deployment of legacy applications. Importantly, the study meticulously evaluates performance metrics — such as computational overhead and CPU utilization — which are crucial in understanding the practical implications of deploying applications on TEE solutions in real-world scenarios (including the costs of the cloud platform setup). The hardware and software configuration used in our study was selected based on typical TEE use cases, primarily in cloud environments where TEEs are widely adopted. Our benchmark suite reflects common workloads in confidential computing, including in-memory databases, web services, and machine-learning frameworks. These applications align with real-world scenarios where TEEs provide significant security benefits, ensuring that our findings are relevant and generalizable across different platforms. These benchmarks include both single-threaded and multi-threaded applications, as well as single-process and multi-process workloads, ensuring that the experimental results are applicable across different contexts. By doing so, we are able to provide a realistic and comprehensive assessment of each TEE approach. More precisely, the experimental activity focuses on workloads with different resource usage profiles: *i*) CPU-intensive — *TensorFlow* and *Pytorch*; *ii*) Memory-intensive — *Redis* and *Hashicorp Vault*; *iii*) I/O-intensive — *NGINX* and *NodeJS*.

By thoroughly evaluating the complexity of integration issues and the performance trade-offs of alternative TEE solutions — covering both process-based and VM-based proposals — our study provides practitioners with a framework that can be applied to other contexts for effectively using these technologies for security improvement of legacy software.

The experimental campaign produced the following outcomes:

- VM-based TEEs are faster (i.e. they have smaller execution times), particularly when handling memory- and I/O-intensive workloads, as compared to Process-based TEEs. They are also characterized by a lower overhead in terms of resource usage.
- Although less performing than VM-based TEEs, the overhead of Process-based TEEs is lower in the case of CPU-intensive workloads (as opposed to memory- and I/O- intensive ones). Since the trust model of Process-based TEEs is stronger, Process-based TEE solutions can thus be the right choice for these workloads, because in many setups they represent a good compromise between performance penalty and security improvement.
- In the domain of VM-based TEEs, TDX outperforms SEV in terms of efficiency. We explicitly note that the performance gap between the two security solutions is much larger than performance gap between the respective CPUs.
- In the domain of process-based TEEs, *Gramine-SGX* consistently outshines *Occlum-SGX*, not only in performance but also in terms of resource consumption.

It is important to notice that this study can support the process of compliance with GDPR and CCPA frameworks. These regulations, in fact, require strong data protection measures, including confidentiality during processing. TEEs provide a viable solution, even though achieving compliance involves balancing security guarantees with performance trade-offs. This study provides useful insights on these trade-offs, supporting the decision over different TEE solutions for regulatory compliance.

The remainder of this work is organized as follows. Section 2 gives a background on the TEE solutions and defines the motivation behind our paper. Section 3 presents previous works focused on the evaluation of the TEE approaches that we cover in this paper. Section 4 describes the evaluation methodology and defines the setup used to conduct the experiments. Section 5 presents the experimental campaign. Section 6 makes an overall discussion on the evaluation results giving an insight on the trade-off among security, performance, and practicality. Finally, Section 7 concludes the document.

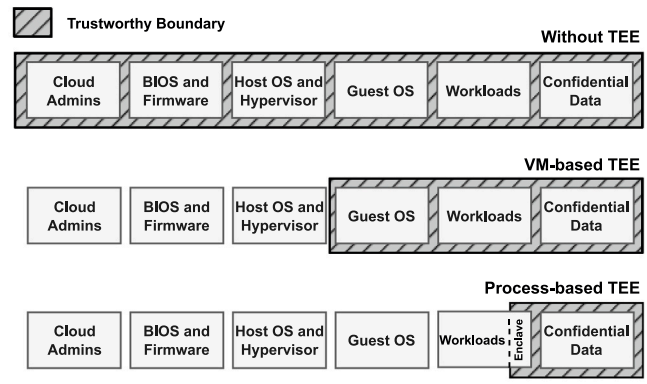


Fig. 1. Trust boundaries of current TEE offerings.

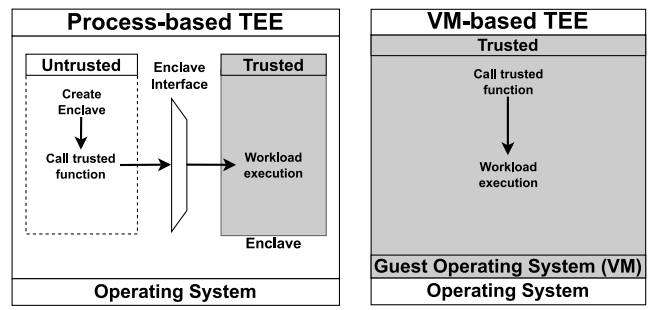


Fig. 2. Process-based vs VM-based TEE.

2. Background and motivation

In the domain of confidential computing, there is a clear-cut division between Process- and VM-based Trusted Execution Environment (TEE) (Fig. 2), whose common goal is ensuring security of data-in-use. In this section, we overview the technologies that are under the magnifying glass of this experimental work. Moreover, we present key aspects that motivate our paper.

2.1. Process-based TEE

In Process-based TEEs, a process is split into two parts: one considered secure (trusted) and the other considered not secure (untrusted). The secure part is located in encrypted memory, managing sensitive computations, while the non-secure part communicates with the operating system and moves I/O from the encrypted memory to other parts of the system. Data transfer into and out of this secure memory zone is tightly regulated, with stringent controls over the data size and type that is allowed to cross the boundaries. Ideally, data transferred to or from the encrypted memory should be encrypted during transit and only decrypted within the TEE, ensuring that it is only accessible to the software operating within the TEE. The widely adopted Process-based TEE for server-side security is Intel Software Guard eXtension (SGX) (McKeen et al., 2016).

SGX enables the creation of *secure enclaves* within the processor, isolating sensitive code and data from the rest of the system thanks to the extension of Intel's Instruction Set Architecture (ISA) by 18

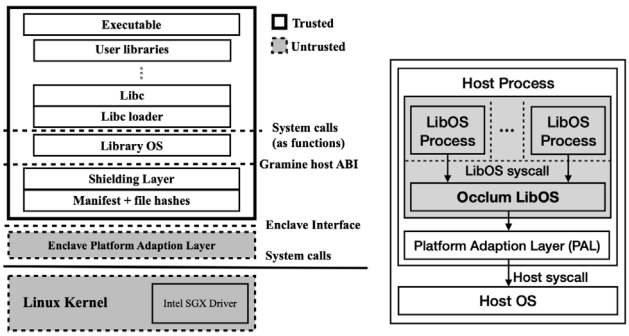


Fig. 3. Gramine (Tsai et al., 2017) (left) and Occlum (Shen et al., 2020) (right) Architecture.

new instructions. Sensitive code and data are stored in the *Enclave Page Cache* (EPC), a specific 128 MiB/256 MiB (for SGX v1 or v2) section of memory set aside during the system startup for storing the code and data of enclaves. Any attempt to access an enclave's page outside of the EPC results in a page fault. The SGX driver collaborates with the CPU to determine which pages to remove from the cache. The memory encryption engine (MEE) ensures that communication between the CPU and system memory remains secure, and it is also responsible for preventing tampering and providing replay protection. An enclave can only run in user mode (*ring3*) since the host OS is considered untrusted. This means that *system calls* cannot be invoked from inside the TEE. The only way to execute them is through well-defined interface calls outside the enclave. An important SGX feature for verifying the integrity and security of an enclave is the attestation mechanism. SGX supports a local attestation, used for communication between enclaves on the same platform, and a remote attestation used for demonstrating trustworthiness to external entities.

Developing applications for SGX can be challenging because the application must be refactored into trusted (within the enclave) and untrusted components. This porting requires careful design to ensure security and can be a complex and time-consuming process. This is where SGX runtimes came to the rescue. They act as intermediate layers that abstract away the complexity of SGX and allow developers to run applications within SGX enclaves with much fewer changes. By using these runtime environments, developers can more easily take advantage of the security benefits of SGX, allowing sensitive or critical applications to be deployed in potentially untrusted environments.

The common approach adopted by these runtimes to enable quasi-transparent porting is to execute system calls directly inside the enclave via a *Library OS (LibOS)*, i.e., a new paradigm trend where kernel functions are available to user space (*ring3*) programs in a form of a library. There are several runtime environments for Intel SGX (e.g. *SCONE, Gramine, Occlum, SGX-LKL*). In this work, we focus on *Gramine* and *Occlum* because they stand out among the open-source solutions as the widely adopted ones (Anon, 2024).

2.1.1. Gramine-SGX

Gramine (Tsai et al., 2017) is a runtime coming with a lightweight library OS, which facilitates the use of dynamically loaded libraries and runtime linking. It stands out as one of the runtimes that fully accommodate fork/clone/execv system calls, which are essential for multi-process abstraction, thereby supporting a wide spectrum of applications. A distinctive attribute of Gramine (Tsai et al., 2017) is its ability to secure dynamic loading, allowing users to incorporate any library into an enclave while ensuring the integrity of the libraries. It enables the safe execution of any binaries, such as those using *glibc* with dynamically linked libraries within the enclave. To do this, a user of Gramine must create a *manifest* detailing all the trusted libraries and data files employed within an enclave and then sign this manifest to safeguard its integrity before executing the chosen binary on SGX. Gramine provides a basic set of system calls in its capacity as a LibOS, which can be processed rapidly due to the low interaction with the host OS. Alternatively, system calls that are not supported by the library OS are meticulously handed over to the host OS. This handover necessitates exits from and re-entries into the enclave, leading to substantial performance costs. Moreover, because the host OS is considered untrustworthy in the SGX security framework, Gramine also verifies the host OS's responses. Hence, system calls that are passed to the host OS incur greater costs compared to those that the library OS can emulate.

2.1.2. Occlum-SGX

Occlum (Shen et al., 2020), while sharing similarities with Gramine as a runtime environment, owes its spread and adoption to its strong community support. Being an open-source project, Occlum benefits from contributions from a range of developers, which helps in its development and maintenance. This community involvement can be important for ensuring the tool stays updated and relevant. This system introduces Software Fault Isolation-Isolated Processes (SIPs) within a LibOS in an enclave's single address space. Software Fault Isolation (SFI) is a technique for sandboxing untrusted modules in different domains. The novel aspect of this proposal is the *Memory Protection Extensions-based (MPX), Multi-Domain SFI (MMDSFI)*, which supports an unlimited number of domains without restrictions on their addresses and sizes. This allows for enhanced intra-enclave isolation, including isolation between processes and between a process and the LibOS. To ensure the security and compliance of these isolation mechanisms, an independent binary verifier called the *Occlum verifier* is introduced. This verifier statically checks ELF binaries to ensure they adhere to MMDSFI's security policies.

2.2. VM-based TEE

VM-based TEEs foresee that an entire VM memory is encrypted using keys sealed in the hardware, which prevent interference by a malicious VMM. Current technologies such as Intel TDX and AMD SEV provide dedicated ephemeral encryption keys for each VM, thus also protecting the VMs from each other.

2.2.1. AMD SEV

AMD SEV (Secure Encrypted Virtualization) (Kaplan et al., 2016) is a security feature in AMD EPYC processors, which utilizes AMD Secure Memory Encryption (SME) and AMD Virtualization (AMD-V) for cryptographically separating VMs from the hypervisor. Each VM gets a distinct, temporary AES key for encrypting memory during operation. The AES mechanism in the processor's memory controller handles encryption and decryption of data to and from the main memory. These keys for each VM are overseen by the AMD Platform Security Processor (PSP). A specific bit (C-bit) in physical addresses is used to encrypt memory pages. SEV also offers remote attestation, enabling VM owners to check the integrity of VMs and the SEV platforms. The PSP creates an attestation report, signed by an AMD-certified key, which VM owners can authenticate along with platform and guest measurements. AMD has introduced three versions of SEV: the first only secures VM memory confidentiality; the second, SEV-ES (Encrypted State), additionally safeguards CPU register states during transitions with the hypervisor; the third, SEV-SNP (Secure Nested Paging), further protects against memory attacks like corruption, replaying, and remapping.

2.2.2. Intel TDX

Intel Trust Domain Extensions (TDX) (Anon, 2023a), as part of the 4th Generation Intel Xeon Scalable Processor, is built using a combination of Intel Virtual Machine Extensions (VMX) ISA extensions, multi-key, total memory-encryption (MKTME) technology, and a CPU-attested, software module. In addition to these technologies, TDX leverages the Intel SGX for what concerns the attestation of Trusted Domains (TDs). Intel TDX enhances the security of TDs by offering protection against certain types of attacks that involve physical access to platform memory. This includes protection against offline attacks, like DRAM analysis, which encompasses cold-boot attacks, as well as active attacks on DRAM interfaces. These active attacks might involve intercepting, altering, moving, splicing, or creating aliases for memory contents. However, Intel TDX does not provide a defense against the replay of memory content through physical attacks. Confidentiality and Integrity of Memory and CPU state is achieved by excluding elements such as firmware, software, devices, and cloud platform operators from the trusted computing base (TCB), giving workloads more secure access to CPU instructions and other technologies. This capability is independent of the cloud infrastructure used to deploy the workload. Remote attestation is another feature provided by TDX, enabling the validation of a workload's environment and the security integrity of the TCB.

2.3. Motivation

As the commercial offering of TEE has expanded over the years, it has become challenging for security engineers and decision-makers to select the right solution matching their requirements. Understanding the performance implications of TEEs is essential. This necessity stems from the need to balance security features with system efficiency, ensuring that the implementation of TEEs does not hinder system performance. Performance metrics such as computational overhead, resource utilization, and impact on response times are critical in determining the viability and appropriateness of TEEs in various operational contexts. Comprehensive knowledge of these aspects enables informed decisions about deploying, configuring, and optimizing TEEs, thus ensuring robust security without compromising on performance. Furthermore, the higher resource usage given by TEEs also entail higher expenses for cloud deployments. Overall, our paper wants to provide arguments for supporting three questions that TEE engineers typically face:

- *How does it impact the performance of the application?*
- *How does it impact infrastructural costs?*
- *How does it impact the customer security perception?*

Given the diversity of solutions, we believe it is important to provide an insight into the performance of the most popular approaches. Each approach has unique characteristics in terms of design, operation, and performance implications. A comparative analysis is essential to understand the trade-offs and benefits of each method. Different applications may have varying requirements. For instance, a blockchain application might prioritize integrity and isolation, a fin-tech application might focus on performance, and a critical infrastructure application might wonder about reliability. A comparative research can help stakeholders select the most appropriate TEE approach for their specific use case. In our analysis, we did not take power consumption into account since — as demonstrated by Göttel et al. (2018) — there are no notable differences among the evaluated TEE approaches.

The balance between security and usability is an age-old challenge. Transparent security aims to minimize user friction while maximizing protection. Our research work can contribute to designing TEEs that better align with user expectations and application requirements. Last but not least, with regulations like GDPR and CCPA imposing stringent data protection requirements, understanding the performance implications of TEEs can assist organizations in making informed decisions that comply with legal standards.

3. Related work

Several works have analyzed the security guarantees, performance trade-offs, and applicability of different TEE technologies across various computing paradigms. This section provides an overview of existing research that compares and evaluates TEEs in terms of performance, security, energy efficiency, and methodological approaches.

Chaudhuri et al. (2025) provide a survey on securing and preserving privacy in GPU-based machine learning inference within TEEs. The study reviews TEE implementations across CPUs, GPUs, and edge devices, discussing security mechanisms such as adversarial attack detection and input pre-processing. The work identifies research gaps in adapting these methodologies across different execution environments, highlighting the need for more universal security approaches. Their methodology involves a systematic literature review, categorizing techniques based on adversarial defense mechanisms, privacy-preserving methods such as homomorphic encryption and differential privacy, and model stability improvement strategies.

Mofrad et al. (2018) compare Intel SGX and AMD's memory encryption technologies (SME and SEV). The study finds that Intel SGX provides strong memory integrity protection, making it suitable for small security-sensitive applications. However, AMD SEV, while lacking memory integrity guarantees, offers a larger amount of secure memory, making it preferable for complex workloads. Their benchmarking approach involves developing benchmark applications compatible with both platforms using standard C/C++ libraries. The benchmarks focus on floating-point intensive workloads, memory encryption engine performance, and secure data provisioning.

Göttel et al. (2018) conduct an empirical study on the security, performance, and energy trade-offs of hardware-assisted memory protection mechanisms, specifically Intel SGX and AMD SEV. They report that AMD SEV has a negligible performance impact, whereas Intel SGX introduces significant overhead. Furthermore, Intel SGX exhibits higher energy consumption compared to native execution, whereas AMD SEV remains energy efficient. To evaluate these trade-offs, the authors implement a publish/subscribe system on a Redis key–value store, incorporating well-known open-source libraries to ensure reproducibility. Their methodology measures performance under different encryption settings.

Akram et al. (2021) analyze the performance of scientific computing workloads in TEEs, specifically Intel SGX and AMD SEV. The study finds that SEV, when configured properly, can operate with minimal overhead in high-performance computing (HPC) environments. In contrast, Intel SGX struggles with HPC workloads due to limited secure memory and poor parallel scaling. Additionally, SEV's virtualization overhead significantly impacts irregular workloads such as graph analytics and I/O-intensive applications. Their methodology evaluates NUMA allocation efficiency in AMD SEV and the impact of virtualization on different workloads through controlled simulations.

Suzaki et al. (2021) introduce TS-Perf, a performance measurement framework designed for fair comparison of TEEs across different CPU architectures, including Intel SGX, Arm TrustZone, and RISC-V Keystone. Their findings indicate that while TEE and non-TEE execution can perform similarly for some workloads, memory encryption and storage access significantly affect TEE performance. The study also uncovers inconsistencies in TEE behavior across implementations, emphasizing the importance of standardized performance evaluation frameworks. TS-Perf uses a compiler-based approach, leveraging the hardware timestamp counter to minimize measurement overhead and enable reproducible benchmarks.

Tan and Mi (2024) evaluate the impact of NVIDIA H100 Confidential Computing (CC) on AI workloads. Their study finds that the encryption overhead in NVIDIA's H100 CC can reduce throughput by up to 93%. To mitigate this issue, they propose an optimized runtime system using multiple encryption workers and asynchronous encryption, reducing the overhead to less than 28.1

Our paper stands out from previous works especially for the evaluation of the new Intel TDX technology. It introduces a novel perspective in the context of TEE research, evaluating the entire spectrum of current approaches for near-transparent porting of applications into TEEs. We conduct a comprehensive comparison of the performance across Intel TDX, AMD SEV, Gramine, and Occlum. Moreover, our work uses a wide set of workloads characterized by completely different resource usage profiles.

4. Methodology

In the following, we outline the hardware and software settings utilized for the experiments, which are analyzed in the rest of the paper.

4.1. Environment

Fig. 4 shows our experimental setup. In order to get results as comparable as possible, we configured our machines hosting the workloads with the same amount of virtual CPUs (vCPUs) cores (4 cores), the same amount of virtual RAM (vRAM) (16 GB), and similar NIC characteristics.

If the workload requires to be stimulated in a client/server topology, there is the need for an external benchmark client to do requests to the server. In this case, in order to prevent interference, a separate VM was deployed. Low-latency channels between the client and server are important to get fair results. Hence, the benchmark VM was deployed in the same *Availability Zone* as the workload VM.

At the time of writing this article, there was no public availability of an Intel TDX machine. Experiments on this technology were conducted using a server offered by Intel for research purposes, which mounts an Intel Xeon Platinum 8480CTDX (2.0 GHz, Turbo at 3.8 GHz). Well-known virtualization tools like QEMU and libvirt are needed for Intel TDX to enhance the confidentiality of active workloads. For the effective operation of a confidential VM, various elements within the virtualization stack must be compatible with TDX hardware. We leveraged private patched versions of Linux kernel, QEMU, and libvirt to deploy a guest TDX VM. Even in this case, the same amount of cores and memory are configured on the mounted QEMU Confidential VM.

At the time of revising the article, instead, some cloud instances with Intel TDX support were announced (Google Cloud[1] and IBM Cloud[2]) or became accessible (Microsoft Azure[3]). This makes it possible to replicate and extend our experiments without requiring dedicated on-premise hardware. Additionally, researchers with access to 4th Gen Intel Xeon Scalable processors (Sapphire Rapids) can deploy TDX-enabled virtual machines locally using QEMU, libvirt, and a TDX-enabled Linux kernel. In fact, Intel recently started to offer its open-source tools, including the TDX-QEMU[4] repository and TDX-KVM[5] patches, to facilitate testing in a controlled environment.

The TDX machine embeds the SGX extension as well. So, we decided to run SGX (i.e., Gramine and Occlum) and *Native* runs — useful to set the baseline for the comparison — in the TDX server. In this way, we can provide a fair comparison of Native, TDX, and SGX. At the same time, we also want to provide information on how SGX performance compares between the old and new generations of CPUs. Hence, we instantiated a Standard DCsv3-series VM, which uses the 3rd Generation Intel Xeon Scalable (Icelake) 8370C (2.9 GHz, Turbo at 3.5 GHz) with the SGXv2 capabilities (i.e., larger EPC and support for dynamic memory allocation (EDMM)).

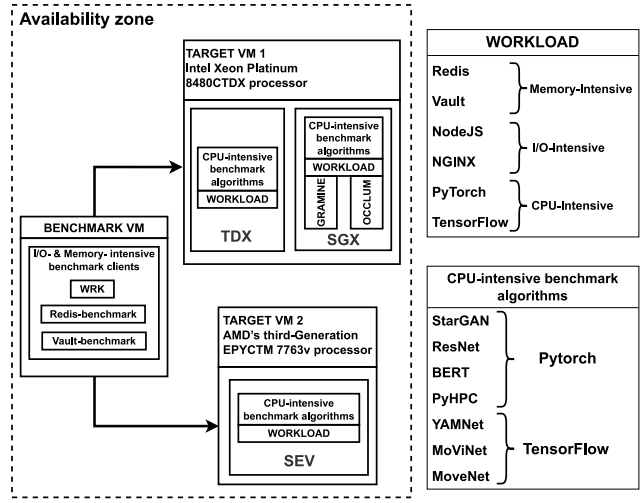


Fig. 4. Experimental Setup.

Experiments on AMD SEV leveraged a Standard DCasv5-series VM, which uses AMD's third-Generation EPYCTM 7763v processor (2.5 GHz, Turbo at 3.5 GHz). We selected this server because it is one of the best-performing SEV-enabled AMD machine, which is comparable with the Intel machine. To ensure a controlled evaluation environment and minimize interference from cloud infrastructure variability, we conducted our experiments using a Metal-as-a-Service (MaaS) solution. This approach allowed us to deploy TDX, SEV, and SGX on dedicated hardware, eliminating factors such as noisy neighbors, hypervisor scheduling overhead, and shared resource contention that are common in multi-tenant cloud environments.

In terms of software, all VMs were configured with Ubuntu 22.04.3 LTS. We use the latest versions of *Gramine-SGX* (v1.6), and Occlum (v0.30.0). The SGX SDK software stack relies on version 2.22.

4.2. Managing CPUs with different clock frequencies

A critical challenge in our comparative analysis was the variation in CPU clock frequencies across the testbed. The Intel 8480C-TDX (used for Native, TDX, and SGX tests) operates at 2.0 GHz (Turbo 3.8 GHz), while the AMD EPYC 7763v (used for SEV) runs at 2.5 GHz (Turbo 3.5 GHz). These differences in base and turbo frequencies can skew performance measurements, making it difficult to isolate the impact of the TEE technology itself from inherent CPU performance characteristics. If raw execution times were directly compared: *i*) SEV could appear significantly slower than TDX, but this might be due to the inherent CPU performance gap rather than the TEE overhead; *ii*) TDX could seem more efficient than SEV, even if the observed speedup results from a higher single-thread performance of the Intel CPU rather than the efficiency of the TEE mechanism itself; *iii*) The impact of SGX and TDX on Intel CPUs could be misinterpreted, since the performance of an SGX workload on an older Intel CPU (8370C) may not align with the performance on a newer Intel CPU (8480C), creating discrepancies in the comparison. By normalizing execution times, we ensure that any differences in performance are attributed solely to the TEE overhead rather than hardware disparities.

To normalize performance data, we used publicly available microbenchmark data (Anon, 2023b,c,d) that measure single-threaded and multi-threaded performance across different CPUs under standardized conditions. These benchmarks provide a relative performance

[1] https://community.intel.com/t5/Blogs/Products-and-Solutions/Security/Intel-and-Google-Cloud-Announce-Confidential-VMs-for-the-Masses/post/1634718

[2] https://community.ibm.com/community/user/cloud/blogs/meryl-veramonti/2025/01/16/intel-tdx-beta

[3] https://learn.microsoft.com/en-us/azure/virtual-machines/sizes/general-purpose/dcesv5-series?tabs=sizebasic

[4] https://github.com/intel/tdx-qemu

[5] https://github.com/intel/tdx-module

factor between CPUs, which we applied as a correction factor. The normalization process followed these steps: *a*) obtain baseline performance benchmarks for the tested CPUs, including integer and floating-point performance; *b*) Determine the relative performance difference between Intel 8480C and AMD 7763v, focusing on workloads similar to those in our study; *c*) Apply a scaling factor to the execution times, compensating for raw CPU performance differences while preserving the TEE-induced overhead. Validate results with an additional control experiment, ensuring that normalized values align with expected theoretical performance. For example, microbenchmark results indicate that the AMD 7763v is approximately 40.7% slower than the Intel 8480C in single-threaded workloads. By adjusting SEV execution times downward by this factor, we derive a more realistic comparison of SEV vs. TDX.

Without normalization, raw execution times would have unfairly penalized SEV, as it runs on a slower CPU. Conversely, TDX would have appeared faster than SEV, potentially misrepresenting its actual efficiency. The adjusted results allow us to confidently state whether performance differences arise from the TEE mechanism or from hardware disparities.

4.3. Workloads

TEE adoption is often constrained by performance overhead, which can vary depending on workload characteristics. To ensure diversity and comprehensiveness, we selected workloads that stress different resource types: *i*) CPU-intensive workloads test the computational efficiency of TEEs, revealing the impact of enclave execution on compute-heavy applications; *ii*) Memory-intensive workloads evaluate data access latency, critical for TEEs with memory encryption or limited EPC sizes. *iii*) I/O-intensive workloads assess performance in applications with high network and disk interactions, where TEEs may introduce system call overhead. This categorization aligns with real-world usage of TEEs in cloud, finance, and machine learning applications, ensuring that our findings apply to diverse deployment scenarios.

CPU-Intensive Workloads. For tasks that demand substantial CPU resources, particularly for computation-intensive processes, we selected the following workloads:

- *TensorFlow*[6]: This is a widely used framework in the field of deep learning. We specifically focused on running machine learning inference algorithms, which are known for their high CPU usage due to complex calculations.
- *PyTorch*[7]: Another deep learning framework, PyTorch is renowned for its efficiency in performing computations that require significant CPU power.

Memory-intensive Workloads. For workloads that predominantly consume memory resources, we included:

- *Redis*[8]: Known for its efficiency as an in-memory key–value store, we utilized Redis for operations like deep in-memory scanning, and typical SET and GET commands, which are memory-intensive.
- *Hashicorp Vault*[9]: This tool is used for key and secrets management and is known to be memory-intensive, making it a suitable test for our memory workload category.

I/O-intensive Workloads. To evaluate the performance under I/O stress, we selected:

- *NGINX*[10]: A high-performance web server, NGINX is used for serving web content, which is typically I/O-intensive due to the nature of web traffic and data transfer.
- *NodeJS*[11]: Known for server-side scripting, NodeJS applications often involve significant I/O operations, especially when handling multiple concurrent requests.

For each of these workloads, we conducted multiple runs to ensure reliability and accuracy in our results. Specifically, we set a confidence interval of 95% and empirically verified that 12 repetitions of our experiments were enough to achieve the aforementioned target. The outcomes were averaged to account for any anomalies and to provide a more accurate representation of the performance. This rigorous testing methodology allows us to comprehensively assess the effectiveness of TEE approaches across a spectrum of real-world applications, ensuring that our findings are both valuable and applicable to a wide range of scenarios.

4.4. Benchmarks

We employed a variety of benchmarking tools to stimulate the different workloads. It is important to notice that their final configuration — e.g., the number of connections, and the ranges of parallel clients — was obtained empirically after several preliminary experiments aimed at reaching the workload saturation point.

Redis. We used *redis-benchmark*, a tool specifically designed for the REDIS key–value store. It is used to measure the performance of a Redis server by running a series of predefined tests. In our study, we used redis-benchmark to make typical operations of SET and GET, thus evaluating the throughput and latency of the *Redis* server during writing and reading from memory. In terms of configuration, we kept the total number of connections to a fixed value of $100k$ and varied the number of parallel clients from 10 to 1000.

NGINX & NodeJS. The *wrk2* benchmark was adopted to generate a significant load against NGINX and NodeJS. It provides a flexible scripting interface that allows us to simulate different types of HTTP requests, which is crucial for testing the performance of NGINX as a web server and NodeJS in server-side scripting scenarios. By adjusting the number of connections, threads, and the duration of the test, we were able to assess how these servers handle high traffic and concurrent connections. The benchmark was configured with a duration of $30s$, a fixed number of connections to 100, and a varying number of clients ranging from 10 to 16000.

Vault. The *vault-benchmark* tool — specifically designed for Hashicorp Vault — helps in evaluating the performance of Vault in various scenarios, including reading and writing secrets, authentication requests, and other secret management operations. We used the vault-benchmark to determine the throughput and response times of Vault during *static_secret_writes* operations, which is critical for understanding its scalability and reliability in a production environment. The benchmark was tuned with $numkvs = 100$, and a $kvsize$ varying in the range $[10, 800]$.

PyTorch. The benchmarking of PyTorch involved the adoption of built-in algorithms, which range from image processing and classification to natural language processing and high-performance computing simulations. Specifically:

- *StarGAN* (*pytorch_stargan-cpu*) — Image-to-image translations.
- *ResNet* (*phlippe_resnet-cpu*) — Image classification.
- *BERT* (*BERT_pytorch-cpu*) — Natural language processing.
- *PyHPC* (*pyhpc_isoneutral_mixing-cpu*) — Scientific simulations in fluid dynamics and climate modeling.

[6] https://www.tensorflow.org
[7] https://pytorch.org
[8] https://redis.io/
[9] https://www.hashicorp.com
[10] https://nginx.org
[11] https://nodejs.org

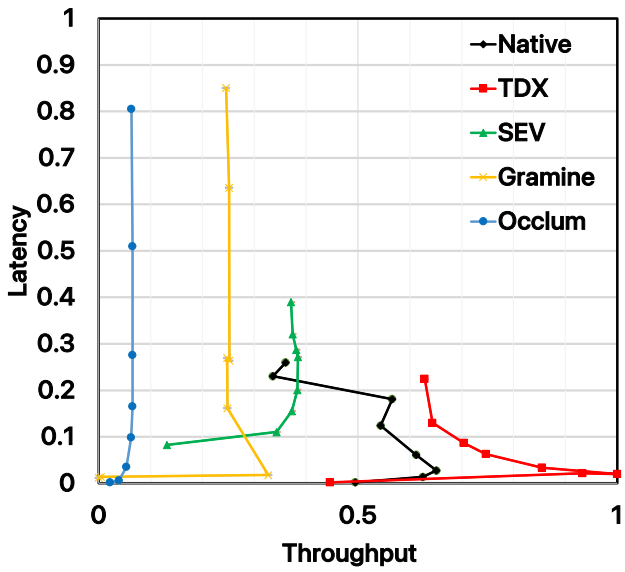


Fig. 5. Redis Performance — Throughput vs Latency.

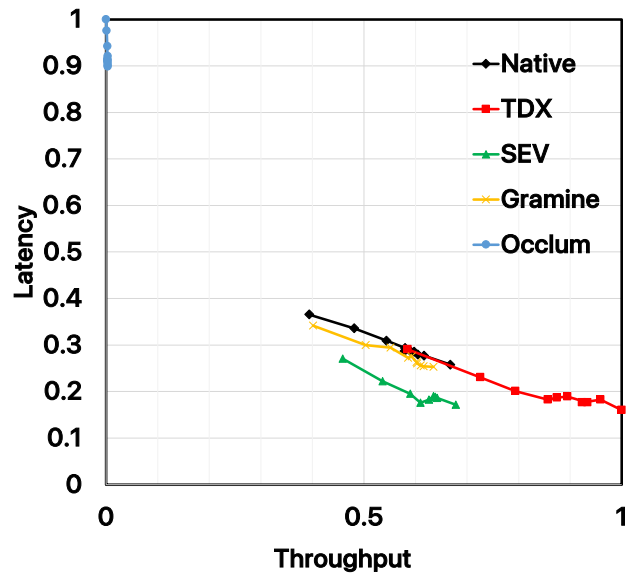


Fig. 6. Vault Performance — Throughput vs Latency.

TensorFlow. Even in this case, we used built-in models to evaluate performance:

- *YAMNet* — a deep learning model designed for audio event detection and classification.
- *MoViNet* (movinet_stream) — a family of models optimized for video understanding, particularly for streaming video analysis.
- *MoveNet* — a cutting-edge model for human pose estimation, known for its speed and accuracy. It is designed to be lightweight and efficient, making it suitable for real-time applications.

5. Experimental results

In this section, we dive into the analysis of results obtained during our experimental campaign. The focus is on interpreting the data collected, evaluating the outcomes against our hypotheses, and understanding the implications of these findings. For CPU-Intensive workloads, we compare the execution time among the different technologies. For Memory and I/O-intensive workloads, instead, we analyze the throughput and latency. In all cases, we also report details of an analysis of the average CPU utilization.

All graphs are normalized as follows:

$$x_{norm} = (x - x_{min})/(x_{max} - x_{min})$$

5.1. Memory-intensive workloads

Fig. 5 shows the graph on *Redis* performance. The x-axis refers to the throughput and the y-axis to the latency. As expected, *Gramine* (yellow line with 'X' marker) and *Occlum* (blue line with star marker) have the worst performance. It can be observed they reported the highest latency and low throughput, although *Gramine* performs better than *Occlum*, which experiences a sharp peak, and then a rapid decrease, suggesting inefficiency, especially at lower throughputs. The *TDX* solution (red line with square marker) has surprisingly the highest throughput and a latency that is comparable with the one observed on the *Native* (black line with circle marker). The *Native* solution has a steady increase in latency with high throughput. Furthermore, we can notice that *Native* and *TDX* have a similar trend.

SEV (green line with triangle marker) has higher latency at the beginning, which sharply increases with a small increase in throughput, indicating a potential bottleneck in handling higher loads. However, it is important to notice that the difference in performance between TDX and SEV depends on two factors: the CPU typology and the security technology itself. The impact of the CPU typology can be obtained using publicly available benchmarks (Anon, 2023b,d), which tell us that the AMD CPU is on average 40.7% slower than the Intel CPU, which hosted the execution of all the other versions of the workloads (Native, TDX, Gramine, Occlum). If we subtract the 40.7% we can argue that the actual overhead of SEV with respect to TDX can be considered of ≈22%.

In Fig. 6, we report the *Vault* performance. The rightest points correspond to the lowest *kvsize*. The increase of *kvsize* causes the decrease of the throughput and a rise in the latency. Even in this case, we observed that TDX provides the highest throughput. It is interesting to notice that SEV has a throughput similar to the *Gramine* and *Native* solutions but at the same time, it has the lowest latency. A different story is Occlum, which experienced very bad performance as can be noticed by the graph highlighting the high latency and the low throughput.

5.2. CPU-intensive workloads

Figs. 7 and 8 show bar graphs depicting the inference time of *PyTorch* and *TensorFlow* workloads, respectively. On the x-axis, we report the different benchmarks used for the evaluation.

For what concerns *PyTorch*, as expected the *Native* environment consistently exhibits the shortest inference times across all benchmarks. *Gramine-SGX* performs exceptionally well, often approaching the performance of the *Native* environment. Unlike the other evaluations, *Gramine-SGX* behaves better than *TDX* across all benchmarks. *Occlum* also exhibits better performance than usual, which is on the same level as TDX. SEV demonstrates significantly longer execution times, with approximately 40% of this increase attributed to inherent CPU characteristic differences.

In the case of *TensorFlow*, the situation is different. TDX experienced a low inference time, which sometimes is even better than the *Native* one. While *Gramine-SGX* and *SEV* reported a slightly higher execution time. *Occlum* was the worst one with a higher inference time, up to 6×.

What becomes evident from these findings is that Process-based TEEs tend to experience only minor performance degradation when subjected to CPU-intensive workloads. In certain instances, their performance closely matches that of VM-based TEEs, while in others, such as with *Gramine-SGX*, they even surpass VM-based TEEs in terms of performance.

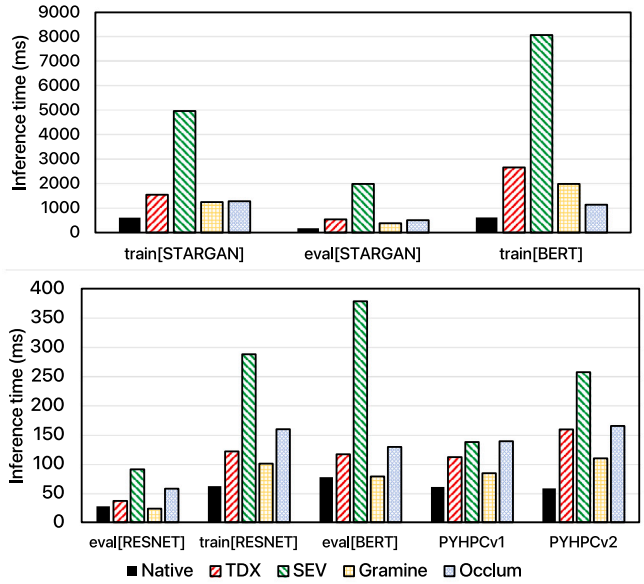


Fig. 7. PyTorch Performance — Inference Time.

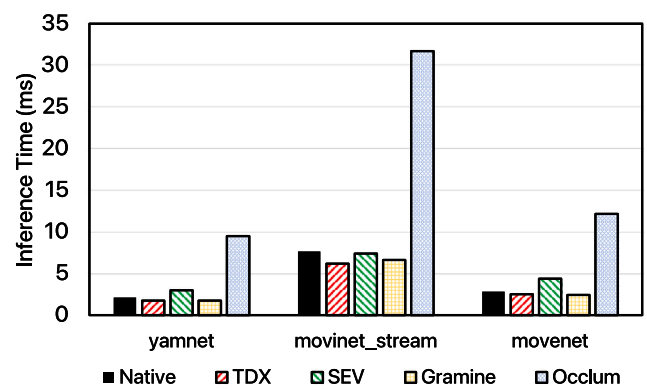


Fig. 8. TensorFlow Performance — Inference Time.

5.3. I/O-intensive workloads

Fig. 9 shows the performance of NGINX. The Native has the highest throughput. The latency remains the lowest across all throughputs. *TDX* shows a significant drop in throughput compared to *Native*, stabilizing just above 0.4.

The latency is low when the throughput is below this point but rises sharply as the throughput increases. The average overhead of *TDX* with respect to the *Native* is 28.6%. SEV has a lower maximum throughput than *TDX*. The latency is in the same range of *TDX* at low throughputs but does not increase as sharply. Again, if we remove the impact of the AMD CPU, we can consider SEV 8.7% slower than *TDX*. *Gramine-SGX* exhibits a similar pattern to *SEV*, but with slightly higher latency across all throughput levels. Even in this case, *Gramine-SGX* behaved better than *Occlum-SGX* but still slower than the VM-based TEEs, reporting an overhead of 67.6%.

Fig. 10 compares the performance of the NodeJS workload. The *Native* has the best performance as before, showing high throughput and low latency. While *TDX* has a slightly higher latency than *SEV*. It is important to highlight that, for this particular workload, both *Gramine-SGX* and *Occlum-SGX* were not able to sustain the same amount of concurrent benchmark requests used for *TDX*, *SEV*, and *Native* (in the range of $[10, 8000]$). The webserver stuck after 500 requests. Hence, a smaller range of requests has been used (i.e., $[10, 500]$). This explains

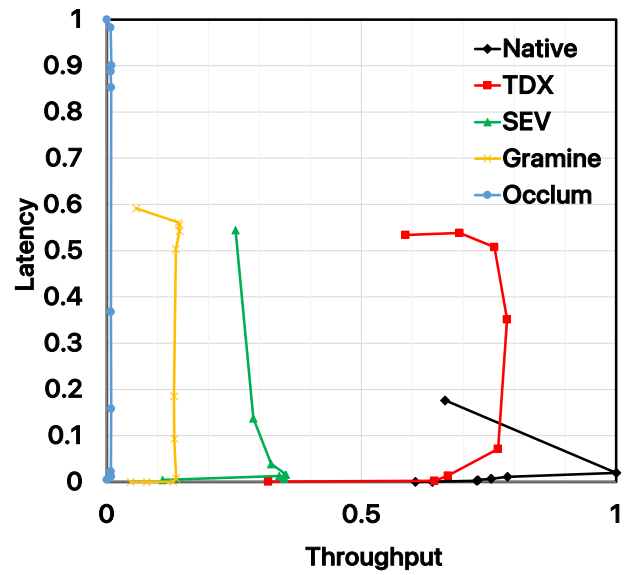


Fig. 9. NGINX Performance — Throughput vs Latency.

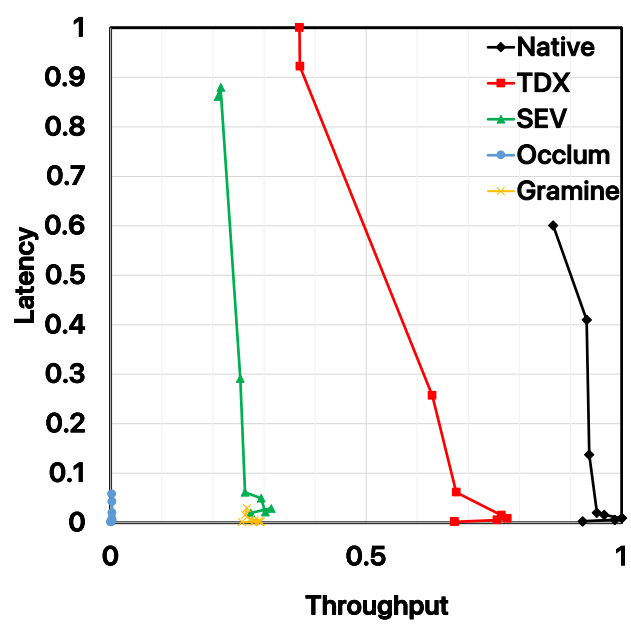


Fig. 10. NodeJS Performance — Throughput vs Latency.

why the latency of *Gramine-SGX* and *Occlum-SGX* is extremely low compared to the other approaches.

5.4. Resource usage

Besides evaluating the speed of our workloads, we are also interested in examining the CPU utilization across the various TEE approaches to better understand the trade-offs between security and computational efficiency.

In Fig. 11 we show the average CPU usage for the different workloads. As expected, the *Native* runs exhibit the lowest CPU usage for all applications. *TDX* shows competitive CPU usage, close to Native levels in most applications. AMD's *SEV* consistently shows higher CPU usage compared to *TDX* and sometimes even higher than *Gramine* and *Occlum*. Both process-based TEEs show variable CPU usage across different applications. *Gramine* generally maintains moderate CPU usage,

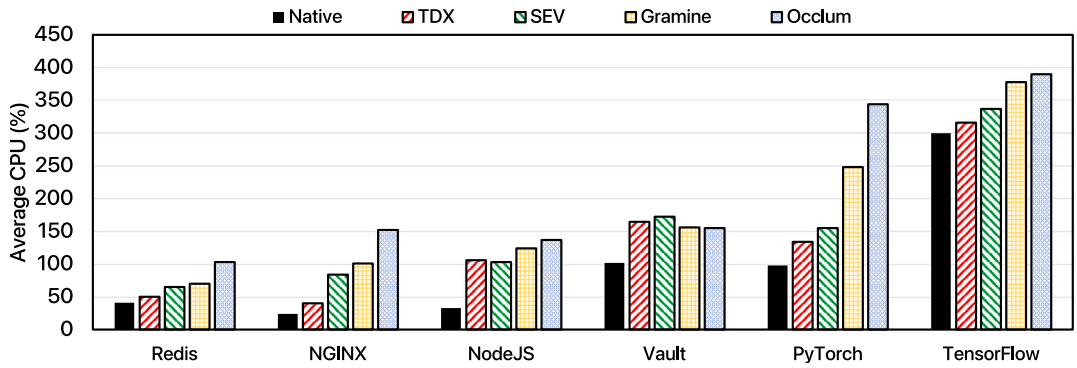
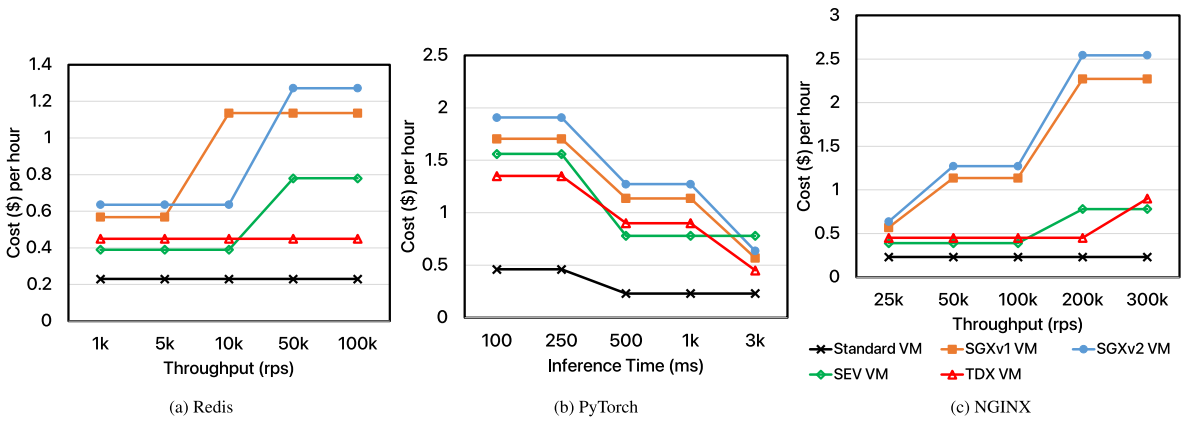


Fig. 11. Average CPU Utilization.



(a) Redis (b) PyTorch (c) NGINX

Fig. 12. Costs of Cloud deployments.

indicating a balanced performance. *Occlum*, however, has notably high CPU usage with PyTorch, which could be indicative of less efficient handling of CPU-intensive workloads or a lack of optimization for this particular application. For Redis and NodeJS, all TEEs have relatively similar CPU usage, with TDX marginally outperforming the others. This suggests that for certain types of workloads, the choice of TEE might not significantly impact CPU efficiency. The PyTorch workload stands out with its high CPU usage in the Native environment, while TDX optimizes this usage considerably. Occlum, however, seems to struggle with this workload. TDX appears to be the most efficient in terms of CPU usage across a range of applications, closely followed by Gramine. SEV tends to have higher CPU overhead, while Occlum's performance is highly variable, performing well in some cases but not in others.

5.5. Impact on cloud service costs

We want to assess how different TEE solutions influence the costs associated with Cloud instances, particularly when striving to achieve predefined performance targets. To facilitate a precise estimation, we first collated the hourly rates from Azure Cloud for various machine types that meet the TEE hardware requirements. We started from the baseline configuration used during our experimental campaign, which consisted of virtual machines equipped with fixed disk size, 4 vCPUs, and 16 GB of vRAM. To evaluate how the variation of vCPUs and vRAM affects performance, we conducted experiments that adjusted these parameters. Notably, certain workloads, such as *Redis*, showed no performance gains from increased core counts due to their single-threaded nature. In such instances, we deploy multiple instances in a clustered configuration under a load balancer to achieve the desired throughput. We point out that the increase of CPU cores in the Cloud VM configuration can only be done in a power of two. Regarding TDX, in the absence of official pricing at the time of this study, we provided an estimated cost. This estimate is based on the announced future availability of confidential TDX VMs on Azure, extrapolating from the current cost structure of similar VM services and accounting for the expected premium that TDX's enhanced security features would necessitate. Regarding SGX, we report costs for two typologies of instances having support for SGX *v1* and *v2* architectures. We selected representative workloads for each category — i.e., *Redis*, *PyTorch*, *NGINX* — and varied benchmark performance targets in a range, which is based on the results obtained during our previous evaluations. We highlight that regardless of the CPU cores, the cost of SGX machines is the highest, followed by the TDX one, and lastly SEV.

Fig. 12 graphically shows the cost per hour of TEE-enabled VMs for each particular selected workload. Regarding *Redis* (Fig. 12(a)), it can be noticed that no increase of CPU cores — thus increase of cost — is needed for Standard and TDX VMs. Contrariwise, SGXv1 machines require more cores starting from $10k$ RPS. While, SGXv2 and SEV increase costs starting from $50k$ RPS. For what concerns *PyTorch* (Fig. 12(b)), we used as a benchmark the training of the BERT algorithm. It can be noticed that SEV is the one that costs more when the inference time must be lower than $3s$, and also it required much more cores when we targeted lower inference times. SGX VMs had a trend similar to the TDX one. This was expected given the performance results we obtained for CPU-intensive workloads. Finally, regarding NGINX, the TDX machine doubles the cores only when a target of $300k$ RPS has to be achieved. While, for the SEV one, it happens with a $200k$ RPS target. SGX machines, instead, require a very high number of cores leading to significantly larger costs. Using $100k$ RPS, the situation gets better.

Table 1
Comparison of approaches for transparent TEE porting.

Technology	TEE type	Security model	Ease of deployment	Performance	Resource usage	Cost implications
Gramine-SGX	Process-based	Strongest isolation (enclave-based); supports dynamic loading	Moderate; requires manifest configuration for application execution	Better than Occlum for most workloads; lower enclave exit overhead	High due to frequent enclave exits but optimized for multi-process applications	Most expensive due to high cloud resource demand
Occlum-SGX	Process-based	Strongest isolation (enclave-based); focuses on intra-enclave isolation	Challenging; strong isolation but requires strict binary verification	Higher overhead due to strict isolation; slower than Gramine	High due to additional isolation layers within the enclave	Most expensive due to high cloud resource demand and additional isolation
Intel TDX	VM-based	Moderate isolation (entire VM protected) with easier deployment	Straightforward; supports unmodified applications	Lower overhead than SGX; strong efficiency for I/O and memory workloads	Lower than SGX; optimized for VM-based workloads	Moderate; lower cost than SGX but higher than SEV
AMD SEV	VM-based	Weaker isolation (entire VM protected) but simplest deployment	Simplest; direct lift-and-shift deployment	Higher overhead than TDX but generally better than SGX	Higher than TDX due to CPU performance limitations	Lowest cost; requires fewer computational resources

6. Discussion: Performance, security, and practicality trade-offs

The performance evaluation highlights key trade-offs between security, efficiency, and practicality when deploying TEE solutions. Our findings (see Table 1) show that VM-based TEEs (TDX, SEV) outperform process-based TEEs (SGX) in I/O- and memory-intensive workloads, while for CPU-intensive workloads, process-based TEEs offer a viable option with security gains at a lower performance cost.

For I/O- and memory-intensive workloads, VM-based TEEs benefit from hardware-assisted virtualization, such as Secure Nested Paging (SEV) and VMX support (TDX), allowing more direct I/O interactions and reduced system call overhead. In contrast, process-based TEEs suffer from frequent enclave exits, leading to performance degradation due to excessive context switches and integrity checks. In the area of process-based TEEs, *Gramine-SGX* consistently outperforms *Occlum-SGX* across all evaluated parameters, including resource consumption. This is due to differences in system call handling, memory management, and execution overhead. Gramine optimizes enclave exits by batching system calls and reducing context switches, while Occlum's multi-process design introduces additional overhead from frequent thread switching and stricter memory isolation. Gramine's lazy memory allocation also reduces SGX page faults, whereas Occlum's fine-grained process isolation increases memory management costs. In the area of VM-based TEE, our results also indicate that TDX performs better than SEV, a difference not entirely attributable to CPU architecture variations. Even after adjusting for CPU-related disparities, TDX demonstrates superior resource efficiency, making it a more scalable option for VM-based confidential computing.

Process-based TEEs like SGX (Gramine, Occlum) enforce strict memory isolation at the enclave level, minimizing the attack surface through encrypted page management and controlled system call interfaces. However, SGX's memory model imposes constraints, requiring enclave-aware modifications and incurring overhead due to asynchronous enclave exits (AEX) and EPC paging costs. These overheads are particularly problematic for memory-intensive workloads, where SGX's limited EPC size leads to excessive page swapping, increasing CPU usage and cloud costs.

VM-based TEEs like SEV and TDX mitigate these issues by encrypting the entire VM, eliminating the need for enclave-specific modifications. SEV-SNP leverages AES-128 memory encryption to prevent hypervisor-based attacks, while TDX isolates VM execution using VMX extensions. However, this approach broadens the trusted computing base (TCB) to include the hypervisor and firmware, introducing additional attack vectors. While VM-based TEEs provide better scalability and easier deployment, they do not offer the same level of fine-grained isolation as SGX, making them more suitable for cloud-native workloads where VM separation is an acceptable security model.

Cloud deployment costs vary significantly between TEE types, driven by CPU and memory requirements. SGX incurs the highest cost, followed by TDX, and then SEV. The cost disparity stems from SGX's reliance on EPC paging, which increases CPU usage for memory-intensive workloads. This issue is less pronounced in TDX and SEV, where memory encryption occurs at the hypervisor level, reducing the need for frequent transitions.

For CPU-intensive workloads, SGX maintains a lower cost overhead, but its reliance on frequent enclave exits increases execution latency. For I/O-heavy applications, SGX VMs required twice the number of CPU cores compared to standard VMs, highlighting the high system call and paging overhead. In contrast, SEV and TDX maintain lower costs for I/O-heavy workloads, as their design minimizes frequent transitions between encrypted and unencrypted memory states.

The practicality of TEE solutions also depends on their maintenance, hardware lifecycle, and security patching, affecting their feasibility in production. TDX and SEV, as VM-based TEEs, rely on cloud providers for hypervisor and firmware updates, ensuring transparent maintenance but potentially causing downtime or requiring VM migrations. Over time, security mitigations may introduce additional performance overhead. SGX, in contrast, demands explicit software maintenance, requiring frequent recompilation and security patching to address vulnerabilities.

The choice between process-based and VM-based TEEs should be driven by workload characteristics and security requirements. SGX offers stronger isolation and verifiable integrity, making it suitable for high-assurance environments but with higher development and deployment complexity. TDX and SEV provide a more practical alternative for general-purpose confidential computing, particularly in cloud-based multi-tenant environments, where VM-based isolation offers a better balance between security, performance, and cost efficiency.

7. Conclusion

This paper provided an in-depth comparative analysis of some of the major solutions for transparent TEE protection of existing applications. We examined the performance and the cost differences of *VM-based TEEs* (specifically, Intel TDX and AMD SEV) against *Process-based TEEs* (i.e., Intel SGX) when used with runtimes such as *Gramine-SGX* and *Occlum-SGX*. The study provides decision-makers with insights useful for understanding which specific TEE solution best suits the requirements/constraints of a given setup. Our evaluation of Intel TDX offers a novel TEE perspective providing insights that could influence its adoption in real-world deployments allowing organizations to lift and shift existing workloads with minimal modifications, reducing the adoption barrier compared to process-based TEEs like SGX. Additionally, TDX demonstrates better resource efficiency than AMD SEV, making it an

appropriate choice for cloud providers and enterprises looking for security guarantees with limited overhead.

It is important to notice that while our setup provided isolation from external cloud influences, real-world deployments could introduce factors such as varying CPU generations, memory speeds, NUMA configurations, and virtualization optimizations, which can affect performance. TDX and SEV, being VM-based, are particularly influenced by hypervisor policies and memory encryption overhead, while SGX, as a process-based TEE, may experience more significant enclave transition costs in high-density multi-tenant environments.

CRediT authorship contribution statement

Luigi Coppolino: Writing – original draft. **Salvatore D'Antonio:** Writing – review & editing. **Giovanni Mazzeo:** Investigation. **Luigi Romano:** Conceptualization.

Acknowledgments

This project has received funding from the European Union's Horizon Europe Research and Innovation Programme under Grant Agreement No. 101070670 (ENCRYPT - A Scalable and Practical Privacy-preserving Framework).

The work made in this paper was also funded by the European Union under NextGenerationEU. PRIN 2022 Prot. n. 202297YF75.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

Akram, A., Giannakou, A., Akella, V., Lowe-Power, J., Peisert, S., 2021. Performance analysis of scientific computing workloads on general purpose TEEs. In: 2021 IEEE International Parallel and Distributed Processing Symposium. IPDPS, pp. 1066–1076. http://dx.doi.org/10.1109/IPDPS49936.2021.00115.

Anon, 2023a. Intel TDX. [Online]. Available: https://www.intel.com/content/www/us/en/developer/tools/trust-domain-extensions/overview.html.

Anon, 2023b. Microbenchmark of intel CPU 8375C. [Online]. Available: https://www.cpubenchmark.net/cpu.php?cpu=Intel+Xeon+Platinum+8375C+%40+2.90GHz.

Anon, 2023c. Microbenchmark of intel CPU 8480. [Online]. Available: https://www.cpubenchmark.net/cpu.php?cpu=Intel+Xeon+Platinum+8375C+%40+2.90GHz.

Anon, 2023d. Microbenchmark of AMD EPYC 7763. [Online]. Available: https://www.cpubenchmark.net/cpu.php?cpu=Intel+Xeon+Platinum+8375C+%40+2.90GHz.

Anon, 2024. Common terminology for confidential computing. [Online]. Available: https://confidentialcomputing.io/wp-content/uploads/sites/10/2023/03/Common-Terminology-for-Confidential-Computing.pdf.

Anon, 2025. An introduction to enarx. [Online]. AvailableW: https://enarx.dev/docs/technical/introduction.

Arnautov, S., Trach, B., Gregor, F., Knauth, T., Martin, A., Priebe, C., Lind, J., Muthukumaran, D., O'Keeffe, D., Stillwell, M.L., Goltzsche, D., Eyers, D., Kapitza, R., Pietzuch, P., Fetzer, C., 2016. SCONE: Secure linux containers with Intel SGX. In: Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation. OSDI '16, USENIX Association, USA, pp. 689–703.

Brossard, M., Bryant, G., El Gaabouri, B., Fan, X., Ferreira, A., Grimley-Evans, E., Haster, C., Johnson, E., Miller, D., Mo, F., Mulligan, D.P., Spinale, N., van Hensbergen, E., Vincent, H.J.M., Xiong, S., 2022. Private delegated computations using strong isolation. Technical Report, Systems Research Group, Arm Research, p. 20. http://dx.doi.org/10.48550/arXiv.2205.03322.

Chaudhuri, A., Shukla, S., Bhattacharya, S., Mukhopadhyay, D., 2025. Secured and privacy-preserving GPU-based machine learning inference in trusted execution environment: A comprehensive survey. In: 2025 17th International Conference on COMmunication Systems and NETworks. COMSNETS, pp. 207–216. http://dx.doi.org/10.1109/COMSNETS63942.2025.10885734.

Coppolino, L., D'Antonio, S., Mazzeo, G., Romano, L., 2019. A comprehensive survey of hardware-assisted security: From the edge to the cloud. Internet Things 6, 100055. http://dx.doi.org/10.1016/j.iot.2019.100055, URL https://www.sciencedirect.com/science/article/pii/S2542660519300101.

Fei, S., Yan, Z., Ding, W., Xie, H., 2021. Security vulnerabilities of SGX and countermeasures: A survey. ACM Comput. Surv. 54 (6), http://dx.doi.org/10.1145/3456631.

Göttel, C., Pires, R., Rocha, I., Vaucher, S., Felber, P., Pasin, M., Schiavoni, V., 2018. Security, performance and energy trade-offs of hardware-assisted memory protection mechanisms. In: 2018 IEEE 37th Symposium on Reliable Distributed Systems. SRDS, pp. 133–142. http://dx.doi.org/10.1109/SRDS.2018.00024.

Hetzelt, F., Buhren, R., 2017. Security analysis of encrypted virtual machines. In: Proceedings of the 13th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments. VEE '17, Association for Computing Machinery, New York, NY, USA, pp. 129–142. http://dx.doi.org/10.1145/3050748.3050763.

Kaplan, D., Powell, J., Woller, T., 2016. AMD memory encryption. pp. 1–12, [Online]. Available: https://developer.amd.com/wordpress/media/2013/12/AMD_Memory_Encryption_Whitepaper_v7-Public.pdf.

McKeen, F., Alexandrovich, I., Anati, I., Caspi, D., Johnson, S., Leslie-Hurd, R., Rozas, C., 2016. Intel® software guard extensions (Intel® SGX) support for dynamic memory management inside an enclave. In: Proceedings of the Hardware and Architectural Support for Security and Privacy 2016. HASP '16, Association for Computing Machinery, New York, NY, USA, http://dx.doi.org/10.1145/2948618.2954331, URL https://doi.org/10.1145/2948618.2954331.

Mofrad, S., Zhang, F., Lu, S., Shi, W., 2018. A comparison study of intel SGX and AMD memory encryption technology. HASP '18, Association for Computing Machinery, New York, NY, USA, http://dx.doi.org/10.1145/3214292.3214301.

Sahita, R., Caspi, D., Huntley, B., Scarlata, V., Chaikin, B., Chhabra, S., Aharon, A., Ouziel, I., 2021. Security analysis of confidential-compute instruction set architecture for virtualized workloads. In: 2021 International Symposium on Secure and Private Execution Environment Design. SEED, pp. 121–131. http://dx.doi.org/10.1109/SEED51797.2021.00024.

Shen, Y., Tian, H., Chen, Y., Chen, K., Wang, R., Xu, Y., Xia, Y., Yan, S., 2020. Occlum: Secure and efficient multitasking inside a single enclave of Intel SGX. In: Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems. ASPLOS '20, Association for Computing Machinery, New York, NY, USA, pp. 955–970. http://dx.doi.org/10.1145/3373376.3378469, URL https://doi.org/10.1145/3373376.3378469.

Suzaki, K., Nakajima, K., Oi, T., Tsukamoto, A., 2021. TS-perf: General performance measurement of trusted execution environment and rich execution environment on intel SGX, arm TrustZone, and RISC-v keystone. IEEE Access 9, 133520–133530. http://dx.doi.org/10.1109/ACCESS.2021.3112202.

Tan, Y., Mi, Z., 2024. Performance analysis and optimization of nvidia H100 confidential computing for AI workloads. In: 2024 IEEE International Symposium on Parallel and Distributed Processing with Applications. ISPA, pp. 1426–1432. http://dx.doi.org/10.1109/ISPA63168.2024.00192.

Tsai, C.-C., Porter, D.E., Vij, M., 2017. Graphene-SGX: A practical library OS for unmodified applications on SGX. In: Proceedings of the 2017 USENIX Conference on Usenix Annual Technical Conference, USENIX ATC '17. USENIX Association, USA, pp. 645–658.